

Laboratório de Concorrência · Aula 10

Arthur Lavagnolli Molina · RA 24031565-2 · Turma ESOF 6S "A"

Gerado em 09/10/2026, 21:11:19 · tempo registrado no cronômetro: 79:43

Preparação: Concorrência em cinco minutos

0.1 Com suas palavras: qual é a diferença entre sincronização de cooperação e de competição? Dê um exemplo do dia a dia (fora da computação) para cada uma.

Sincronização de cooperação é quando um processo depende do outro para sua continuidade, ou seja, o processo 1 espera o processo 2 finalizar para continuar, e vice-versa.

Sincronização de competição é quando 2 ou mais processos precisam acessar o mesmo recurso ao mesmo tempo, gerando a condição de corrida, pois usará o recurso aquele que "chegar primeiro".

Prática 1: A condição de corrida

Previsão: Duas threads somam 1.000.000 cada. Que valor você espera que o programa imprima?

2.000.000

1ª execução (n = 1.000.000)

1028712

2ª execução (n = 1.000.000)

1430049

3ª execução (n = 1.000.000)

1007553

Saída do programa

```
Saída obtida com n = 1.000.000
esperado: 2000000
obtido: 1028712
```

```
esperado: 2000000
obtido: 1430049
```

```
esperado: 2000000
obtido: 1007553
```

```
saída obtida com n = 1000
esperado: 2000
obtido: 2000
```

1.1 Os valores bateram com o esperado? Foram iguais nas três execuções? Por que mudam de uma execução para outra?

Não foram conforme esperado, nas três execuções foram diferentes e mudaram em cada execução devido o escalonamento das threads que fizeram com que a variável mudasse o valor quando o escalonador parou a primeira thread para iniciar a segunda

1.2 Explique, passo a passo, como uma atualização se perde. Use os três passos (ler, somar, gravar) e as duas threads.

um exemplo do que pode acontecer para resultar nesse valor, t1 é iniciado e lê a variável contador com valor 0 por exemplo, e então o escalonador muda para a execução de t2, que lê a variável contador com valor 0 também, em seguida volta para t1 e soma e grava, deixando a variável com valor 1, após isso, executa t2 que irá somar e gravar o que leu que foi 0, incrementando e deixando o valor como 1 também. Neste caso, as duas threads incrementaram porém o valor continuou como 1 no contador

1.3 Com n = 1000, o que aconteceu? Por que isso torna esse tipo de erro perigoso em sistemas reais? Pense em como o programa se comportaria nos testes e em produção.

Devido ao baixo volume de iteração de incremento do loop, ao iniciar a t2, a t1 já terminou de incrementar, fazendo com que a soma finalize de forma correta. Isso é perigoso pois é um resultado subjetivo, condicionado a condição do escalonador, podendo surtir efeitos inesperados no sistema. Um caso como esse pode funcionar normal nos teste e na produção gerar algum erro, devido a incerteza que esse tipo de desenvolvimento sem o controle adequado gera.

Prática 2: Todas as respostas possíveis (Figura 13.1)

Ordens testadas no simulador

AAABBB → 8
ABABAB → 6
AABBAB → 6
BBBAAA → 7
BBAABA → 4
ABAABB → 6
AABBBA → 4

Saída do programa

```
AAABBB → 8
BBBAAA → 7
ABABAB → 6
Ordens possíveis: 20
TOTAL = 4: 9 ordens, por exemplo AABBBB
TOTAL = 6: 9 ordens, por exemplo AABABB
TOTAL = 7: 1 ordens, por exemplo BBBAAA
TOTAL = 8: 1 ordens, por exemplo AAABBB
```

2.1 Quantas ordens são possíveis e quantos resultados diferentes aparecem? Quantas ordens dão o resultado "correto" (8)?

são possível 20 possíveis combinações com 4 resultados possíveis. A ordem do resultado correto é AAABBB

2.2 Escolha uma ordem que dá 4 e explique passo a passo por que o resultado é 4. O que aconteceu com a operação de B?

AABBBA

total + 1 lê = 3

total + 1 calcula = 4

total * 2 lê = 3

total * 2 calcula = 6

total * 2 grava = 6

total + 1 grava = 4

Prática 3: Corrigindo a corrida

Previsão: As três versões fazem a mesma conta. Qual você acha que será a mais rápida? E alguma será incorreta?

Acho que a operação atômica será mais rápida e nenhuma será incorreta.

Saída do programa

Saída da primeira execução:

```
esperado:          20000000
sem sincronizacao: 1008083 em 2 ms
synchronized:      20000000 em 110 ms
AtomicInteger:     20000000 em 62 ms
```

Saída da segunda execução:

```
esperado:          20000000
sem sincronizacao: 1017793 em 2 ms
synchronized:      20000000 em 179 ms
AtomicInteger:     20000000 em 27 ms
```

3.1 Quais versões deram o valor correto? Compare os tempos: por que a versão correta costuma ser mais lenta que a versão sem sincronização?

Ambas, a versão correta costuma ser mais lenta pois precisa fazer o controle de sincronização

3.2 Uma transferência bancária tira dinheiro de uma conta e coloca em outra. Um AtomicInteger em cada saldo resolve? Por quê? O que seria preciso?

Nesse caso não seria possível, pois para esse caso precisaria de sincronização de cooperação, pois envolvem duas contas distintas e a falta dessa sincronização nas contas pode gerar dados inconsistentes. Seria preciso bloquear as duas contas ao mesmo tempo para impedir algumas movimentação inesperada.

Prática 4: Impasse (deadlock)

Previsão: Leia o código: A pega X e depois Y; B pega Y e depois X. As mensagens "A terminou" e "B terminou" vão aparecer?

Não, pois uma fica aguardando a outra finalizar e liberar o recurso

Saídas (antes e depois da correção)

Saída da primeira execução:

```
IMPASSE: 2 threads presas
```

Saída da segunda execução após correção:

```
A terminou
B terminou
sem impasse
```

4.1 Por que as duas threads ficaram presas? Descreva o que cada uma tinha e o que cada uma esperava.

Ficaram presas devido deadlock, que ocorre quando duas ou mais threads esperam o recursos uma da outra e acabam entrando em um impasse.

A tinha x e esperava o recurso y

b tinha y e esperava o recurso x

4.2 Por que pegar os recursos sempre na mesma ordem elimina o impasse? Dê um exemplo de sistema real em que isso poderia acontecer.

O impasse só ocorre quando há uma espera circular, se tudo segue uma fila onde não há um ciclo de dependências, o travamento é evitado porque as threads esperam de forma contínua, garantindo que sempre haja pelo menos um caminho livre para alguém terminar e liberar os recursos.

Prática 5: Mensagens em vez de memória compartilhada

Previsão: São 6 tarefas de 100 ms divididas entre 3 trabalhadores. Quanto tempo você espera que leve no total? E com 1 trabalhador? E com 6?

não respondido

Tempo com 1 trabalhador

600ms

Tempo com 3 trabalhadores

200ms

Tempo com 6 trabalhadores

100ms

Saída do programa

```
trabalhador 1 fez a tarefa 1
trabalhador 1 fez a tarefa 2
trabalhador 1 fez a tarefa 3
trabalhador 1 fez a tarefa 4
trabalhador 1 fez a tarefa 5
trabalhador 1 fez a tarefa 6
1 trabalhadores, 6 tarefas de 100 ms: levou 600ms
```

```
trabalhador 2 fez a tarefa 3
trabalhador 1 fez a tarefa 2
trabalhador 3 fez a tarefa 1
trabalhador 2 fez a tarefa 4
trabalhador 3 fez a tarefa 6
trabalhador 1 fez a tarefa 5
3 trabalhadores, 6 tarefas de 100 ms: levou 200ms
```

```
trabalhador 2 fez a tarefa 3
trabalhador 5 fez a tarefa 6
trabalhador 6 fez a tarefa 1
trabalhador 1 fez a tarefa 2
trabalhador 3 fez a tarefa 4
trabalhador 4 fez a tarefa 5
6 trabalhadores, 6 tarefas de 100 ms: levou 100ms
```

5.1 Explique os três tempos. Por que este programa não precisa de synchronized nem de trava?

Com 1 trabalhador, as 6 tarefas são feitas uma depois da outra: $6 \times 100 = 600$ ms. Com 3, cada um faz 2 tarefas ao mesmo tempo que os outros: $2 \times 100 = 200$ ms. Com 6, cada um faz 1 tarefa e todas rodam juntas: 100 ms. O

programa não precisa de `synchronized` nem de trava porque os trabalhadores não alteram nenhuma variável compartilhada. Eles só recebem as tarefas pelo canal, e o próprio canal garante que cada tarefa vai para um trabalhador só.

Saídas (antes e depois da troca)

```
fatal error: all goroutines are asleep - deadlock!
```

```
goroutine 1 [chan send]:
main.main()
    /box/main.go:9 +0x38
exit status 2
```

```
segunda saída
1
```

5.2 Por que o primeiro programa trava? Por que o canal com espaço para 1 mensagem resolve? (Dica: num canal sem espaço, quem envia espera alguém receber.)

O canal sem buffer exige que quem envia espere alguém receber. A `main` faz `ch <- 1` e fica parada esperando um receptor, mas o receptor seria a própria `main` na linha seguinte, que nunca chega. Todas as goroutines ficam bloqueadas e o Go detecta o deadlock. Com espaço para 1 mensagem, o envio coloca o valor no buffer e não precisa esperar, então a `main` segue e recebe o 1 na linha de baixo.

Prática 6: O compilador como guarda

Previsão: Este programa faz uma thread somar 1 a uma variável de `main`. O que você acha que acontece?

Rust não vai deixar compilar.

Saídas (o erro e o resultado com `move`)

```
error[E0373]: closure may outlive the current function, but it borrows `contador`, which is owned by the current function
  → HelloWorld.rs:7:27
```

```
|
7 |     let t = thread::spawn(|| {
|                               ^^ may outlive borrowed value `contador`
8 |         contador += 1;           // a thread altera a variável de main
|         ----- `contador` is borrowed here
|
```

```
note: function requires argument type to outlive `static`
  → HelloWorld.rs:7:13
|
```

```
7 |     let t = thread::spawn(|| {
|     -----^
8 | |         contador += 1;           // a thread altera a variável de main
9 | |     });
| |     ^
```

help: to force the closure to take ownership of `contador` (and any other referenced variables), use the `move` keyword

```
|
7 |     let t = thread::spawn(move || {
|                               +++++
```

```
error[E0502]: cannot borrow `contador` as immutable because it is also borrowed as mutable
```

```

→ HelloWorld.rs:11:20
|
7 |         let t = thread::spawn(|| {
|           -                -- mutable borrow occurs here
|   -----|
|   |
8 |   |         contador += 1;                // a thread altera a variável de main
|   |         ----- first borrow occurs due to use of `contador` in closure
9 |   |     });
|   |     ----- argument requires that `contador` is borrowed for `'static`
10 |         t.join().unwrap();
11 |         println!("{}", contador);
|           ^^^^^^^^^ immutable borrow occurs here
|
note: requirement that the value outlives `'static` introduced here
→
/rustc/48a229ceaefd4985c50990b14116b6d856af0985/library/std/src/thread/functions.rs:128:14

error: aborting due to 2 previous errors

```

Some errors have detailed explanations: E0373, E0502.
For more information about an error, try `rustc --explain E0373`.

segunda execução

warning: value captured by `contador` is never read

```

→ HelloWorld.rs:8:9
|
8 |         contador += 1;                // a thread altera a variável de main
|         ^^^^^^^^^^
|
= help: did you mean to capture by reference instead?
= note: `#[warn(unused_assignments)]` (part of `#[warn(unused)]`) on by default

warning: value assigned to `contador` is never read
→ HelloWorld.rs:8:9
|
8 |         contador += 1;                // a thread altera a variável de main
|         ^^^^^^^^^^^^^^^^^
|
= help: maybe it is overwritten before being read?

```

warning: 2 warnings emitted

Saída do programa

```

threads.push(thread::spawn(move || {
    for _ in 0..100_000 {
        *c.lock().unwrap() += 1;    // trava, soma e destrava
    }
}));
for t in threads { t.join().unwrap(); }
println!("contador = {}", *contador.lock().unwrap());
}

```

```
510 ms
contador = 400000
```

6.1 Com move, o programa compila mas imprime 0. Por quê? (Dica: move faz a thread levar uma cópia do número.)

Como o contador é um número inteiro (que implementa Copy), o move faz a thread levar uma cópia dele. A thread soma 1 na cópia dela, e a variável da main continua 0. Por isso imprime 0, e o compilador até avisa que o valor capturado nunca é lido.

6.2 Compare com a Prática 1: em que momento o erro foi descoberto em Java e em Rust? Qual é a vantagem e qual é o custo para quem programa?

Em Java, o erro só apareceu na execução, como resultado errado e sem nenhum aviso, e às vezes nem aparece. Em Rust, o erro foi pego na compilação, antes de rodar. A vantagem é que a corrida de dados não chega em produção. O custo é que quem programa precisa entender posse, Arc e Mutex e brigar com o compilador até ele aceitar o código.

Prática 7: Uma thread só: o laço de eventos

Previsão: Em que ordem você acha que as letras A, B, C e D serão impressas?

A, D, C, e B

Saída do programa

```
A
D
C
B
E: timer de 10 ms rodou depois de 300 ms
```

7.1 Em que ordem as letras saíram? Por que B, com tempo 0, saiu depois de D e de C?

Saiu A, D, C, B. Primeiro roda todo o código síncrono (A e D). Depois roda a fila de microtarefas, onde está a promise (C). Só depois roda a fila de tarefas, onde está o setTimeout (B). Mesmo com tempo 0, o setTimeout só entra na fila e espera o código atual e as microtarefas terminarem.

7.2 O timer de 10 ms (linha E) rodou bem depois de 10 ms. Por quê? O que isso significa para um servidor Node.js que atende muitos usuários e faz um cálculo pesado?

Porque o JavaScript tem uma única thread, e a função ocupado prendeu essa thread por 300 ms. O timer já estava pronto em 10 ms, mas só conseguiu rodar quando o laço de eventos ficou livre. Num servidor Node.js, um cálculo pesado trava todos os usuários ao mesmo tempo, porque ninguém é atendido enquanto ele roda. O ideal é dividir o cálculo, usar código assíncrono ou mandar o trabalho pesado para um Worker.

Fechamento: Quem impede a corrida?

Java sem sincronização

Ninguém: há corrida

Java com synchronized

O programador, com sincronização

Go com canais

O projeto: não há memória compartilhada

Rust com Arc e Mutex

O compilador

JavaScript

Só existe uma thread

8.1 Um site de venda de ingressos tem 1 lugar restante, e duas pessoas clicam em Comprar ao mesmo tempo. Qual problema desta atividade pode acontecer? Como você o evitaria?

Pode acontecer uma condição de corrida: os dois pedidos leem que ainda tem 1 lugar livre antes de qualquer um gravar a compra, e o mesmo lugar é vendido duas vezes. Eu evitaria fazendo a verificação e a reserva como uma operação só, com exclusão mútua: uma trava ou uma transação no banco de dados (por exemplo, um UPDATE que só reserva se o lugar ainda estiver livre). Assim, só um dos dois consegue, e o outro recebe a mensagem de esgotado.

8.2 Em uma frase: o que mais surpreendeu você nesta atividade?

O que mais me surpreendeu foi que, no exemplo do livro, o resultado correto era o menos provável, só 1 das 20 ordens possíveis.